Sensitive Data Detection & Compliance Assistant

Project Report — AI Research Innovation Internship Assignment

Submitted to: Proteccio Data — Careers Team
Submitted by: Manas Khanna
Submission Date: July 2, 2026

GitHub: https://github.com/ManasKhanna31/Sensitive-Data-Detection-Compliance-Assistant

---

Live Deployment Link: **https://sensitive-data-detection-compliance-assistant-bxgfneyswgfhbezt.streamlit.app/**

Demo Video Link: **https://drive.google.com/file/d/1_Y4_uGVIIRqj9YrSWKeqktns0tLY99wo/view?usp=sharing**

1. Executive Summary

This project delivers a fully functional, deployed AI-powered application that detects sensitive and confidential information across PDF, TXT, and CSV documents, classifies compliance risk, generates human-readable compliance summaries, and answers natural-language questions about the uploaded content — all built and tested to run without any required paid API.

Beyond the mandatory functional requirements, the submission implements all eight optional bonus features: data masking/redaction, a lightweight RAG-style Q&A pipeline, OCR support for scanned documents, multi-document portfolio analysis, an interactive multi-view dashboard, Docker packaging, a live cloud deployment, and audit logging.

The design deliberately favors deterministic, auditable techniques — regex with structural validators, weighted risk scoring, and TF-IDF retrieval — over a black-box LLM call for the core detection and classification logic. This is a considered choice explained in detail in Section 4, and reflects how real-world DLP (Data Loss Prevention) and compliance tooling is typically built.

2. Objective

Build an AI-powered application that can:

- Accept document uploads in PDF, TXT, and CSV formats
- Detect sensitive and confidential information across ten distinct categories
- Classify each document's overall risk level as Low, Medium, or High
- Generate a structured compliance and security summary with remediation guidance
- Allow the user to ask natural-language questions about the uploaded document

3. Features Implemented

3.1 Mandatory Functional Requirements

✓ Document upload — PDF, TXT, CSV, all tested end-to-end

✓ Sensitive data detection — Aadhaar, PAN, email, phone, credit card, bank account/IFSC, API keys, passwords, AWS keys, employee IDs, confidential-business markers (10 categories)

✓ Risk classification — Low / Medium / High, via a transparent weighted + log-scaled scoring model

✓ AI-generated compliance summary — observations, security risks, and per-category remediation steps

✓ Conversational Q&A — answers the four example questions from the brief, plus open-ended questions via retrieval

3.2 Bonus Features — All 8 Implemented

✓ OCR support — Tesseract-based fallback automatically triggers for scanned/image-only PDFs with no text layer

✓ Data masking/redaction — downloadable, safe-to-share redacted version of any document

✓ RAG implementation — TF-IDF + cosine-similarity retrieval over document sentences, a lightweight RAG pipeline requiring no embedding API

✓ Multi-document support — batch upload with a portfolio-level risk overview and per-document drill-down

✓ Dashboard/UI improvements — multi-view layout, bar charts, metrics, chat interface, downloadable reports

✓ Dockerization — a single Dockerfile with OCR system dependencies pre-installed

✓ Deployment — live on Streamlit Community Cloud

✓ Audit logging — every scan appended to a structured JSONL log, viewable in-app

4. Architecture Overview

The application follows a linear, single-responsibility pipeline. Each stage is an independently testable Python module with no framework dependency, which is what allowed every module to be unit-tested outside of Streamlit during development.

Stage	Module	Responsibility
1. Ingestion	document_loader.py	Normalizes PDF / TXT / CSV into plain text; OCR fallback for scanned PDFs
2. Detection	detectors.py	Regex + structural validators (Luhn, Aadhaar rule, IFSC format) find sensitive entities
3. Risk Scoring	risk_classifier.py	Weighted, log-dampened score maps detections to Low / Medium / High Risk
4. Summarization	summarizer.py	Template-driven compliance observations, security risks, remediation steps
5. Q&A	qa_engine.py	Rule-based intent layer + TF-IDF retrieval layer answer natural-language questions
6. Redaction	redaction.py	Produces a masked, safe-to-share version of the document
7. Audit	audit_logger.py	Appends every scan to a structured JSONL log

Data flow: upload → text extraction (with OCR fallback) → regex detection with validators → weighted risk scoring → compliance summary generation → all outputs feed a QAEngine that answers questions from structured data (fast path) or via TF-IDF retrieval (fallback).

5. AI/ML Approach Used

The project combines rule-based NLP with a lightweight retrieval-based QA approach rather than depending on a paid LLM API, so the full pipeline runs offline at zero cost and is trivially reproducible by an evaluator.

- **Detection — regex + structural validation.** Each entity type pairs a regex pattern with a validator function wherever a checksum or structural rule exists: the Luhn algorithm for card numbers, Aadhaar's "never starts with 0 or 1" rule, and IFSC's 4-letter + 0 + 6-alphanumeric structure. This mirrors how production DLP tools reduce false positives beyond naive regex matching.
- **Risk scoring — weighted, log-dampened.** Each category carries a hand-assigned severity weight (a leaked password scores 10; an email address scores 3). Raw scores are log-scaled so high detection

volume of low-risk items cannot outrank a document containing a single exposed credential — reflecting how real compliance risk is dominated by the most sensitive item present, not sheer count.

- **Question answering — two-layer design.** An intent layer matches common compliance questions directly to structured detection data for guaranteed, hallucination-free answers. A retrieval layer (TF-IDF + cosine similarity over document sentences) handles open-ended questions that don't match a known intent — a minimal RAG pipeline needing no embedding API. An optional LLM upgrade path is wired in (activated only if an API key is present) but is never required.
- **OCR — conditional fallback.** Tesseract OCR only triggers when standard PDF text extraction yields near-empty output, keeping normal PDFs fast (sub-millisecond) while still supporting scanned documents.

6. Technology Stack

Layer	Technology
Interface	Streamlit
Detection	Python re (regex) + custom validators
Risk Scoring	Custom weighted model (math.log2 dampening)
Q&A Retrieval	scikit-learn TfidfVectorizer + cosine similarity
Document Parsing	pypdf, pandas
OCR	pytesseract + pdf2image (Tesseract OCR engine)
Optional LLM	OpenAI-compatible chat completion API (opt-in only)
Packaging	Docker
Deployment	Streamlit Community Cloud

7. Evaluation Criteria — Self-Assessment

Criterion	How this project addresses it
Functionality	All 5 mandatory requirements + all 8 bonus features implemented and tested end-to-end
AI/ML Understanding	Two-layer Q&A design, checksum-validated entity detection, log-dampened risk scoring — each with documented reasoning
Problem-Solving Ability	Six real challenges documented with concrete fixes (see Section 8)
Code Quality	Modular single-responsibility files, docstrings throughout, graceful error handling at every I/O boundary
Documentation	README covers setup, architecture, AI/ML approach, challenges, future improvements, plus this report
Security & Compliance Thinking	Structural validators, redaction, audit logging, per-category remediation guidance, synthetic-data disclaimer
Innovation	Log-scaled risk weighting; hybrid rule+retrieval Q&A with zero required API cost; conditional OCR fallback

8. Challenges Faced

- **Regex recall vs. false positives:** a naive 12-digit pattern matches Aadhaar numbers, bank account numbers, and invoice numbers alike. Solved with structural validators plus a priority/overlap system so specific patterns (e.g. IFSC) claim a text span before generic catch-alls do.
- **CSV/table context:** PII detectors are built for free text; CSVs needed to be flattened row-by-row while preserving enough surrounding context for detection results to remain meaningful.
- **Risk scoring calibration:** a plain sum of weights over-penalizes documents with many low-risk items. Log-dampening the raw score fixed this; thresholds remain heuristic and would benefit from calibration against labeled real-world examples.
- **Q&A without a guaranteed LLM key:** a two-layer (intent + TF-IDF retrieval) system keeps the app fully functional and demoable with zero paid dependency, while leaving a clean integration point for an LLM.
- **OCR accuracy on scanned documents:** Tesseract occasionally misreads characters in low-resolution scans, which can cause a regex detector to miss a match it would catch on clean text — an inherent OCR-engine limitation, not a detection-logic bug.
- **Multi-document risk ranking:** when two documents share a risk tier, the portfolio view needed the underlying weighted score as a secondary sort key to correctly identify the actually-riskier document — tier alone was not precise enough.

9. Future Improvements

- Swap/augment regex detection with a trained NER model (e.g. a spaCy custom pipeline or fine-tuned transformer) for higher-precision entity detection, especially for unstructured fields like general confidential business language.
- Proper vector-store-backed RAG (FAISS/Chroma) for large multi-page documents, where sentence-level TF-IDF may miss cross-sentence context.
- Role-based access control and encrypted storage for the audit log in a real deployment.
- Fuzzy/approximate matching post-OCR to compensate for character misreads on scanned documents.
- Cross-document Q&A that can answer questions spanning the entire uploaded portfolio, not just the currently selected document.

10. Submission Links

GitHub Repository: https://github.com/ManasKhanna31/Sensitive-Data-Detection-Compliance-Assistant

Live Deployment: https://sensitive-data-detection-compliance-assistant-bxgfneyswgfhbezt.streamlit.app/

Demo Video: https://drive.google.com/file/d/1_Y4_uGVIIRqj9YrSWKeqktns0tLY99wo/view?usp=sharing

---

Disclaimer: *All data in the included sample files is synthetic and fabricated for demonstration purposes only. This tool is a prototype built for a technical assignment and is not a certified compliance or legal product.*